

# Funciones y pasajes de parámetros

## Clase 10

# Función

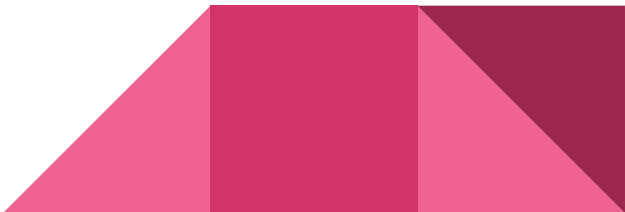
Una función es un bloque de código independiente que realiza una operación. Es reutilizada, lo que ayuda con la legibilidad y modularización de un programa. Puede recibir múltiples parámetros y devolver distintos valores.

```
def saludo(nombre):  # nombre es un parámetro  
    print(f'Hola, {nombre}')
```

```
saludo('Diego')      # 'Diego' es un argumento
```

```
def saludo(nombre):  
    return f"Hola, {nombre}"
```

```
print(saludo('Juan'))
```



# Pasaje de parámetros

Se debe tener cuidado al enviar variables como parámetros ya que podríamos ocasionar modificaciones indeseadas a estas.

```
def modificar(a):
```

```
    a = 10
```

```
a = 2
```

```
modificar(a)
```

```
print(a)    # 2
```

```
def modificar_lista(lis):
```

```
    lis[0] = 'cambio'
```

```
lista = [0, 1]
```

```
modificar_lista(lista)
```

```
print(lista)    # ['cambio', 1]
```



# Pasaje de parámetros

En Python la referencia a los argumentos va a depender si son mutables o inmutables.

Los cambios en los inmutables no se verán reflejados en el programa principal.

Los mutables solo se modificarán si sus elementos son alterados, mas no si el parámetro en si es reasignado.



# Posición de argumentos y valor por defecto

```
def funcion(a, b, c):  
    print(a, b, c)
```

```
funcion(1, 2, 3)    # 1 2 3
```

```
funcion(c=1, a=2, b=3) # 2 3 1
```

```
funcion(1, c=2, b=3)    # 1 3 2
```

```
def funcion2(a, b, c, d=4): # valor por defecto  
    print(a, b, c, d)
```

```
funcion2(1, 2, 3)      # 1 2 3 4
```

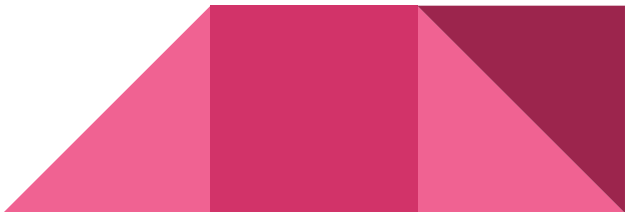
```
funcion2(1, 2, 3, 9)   # 1 2 3 9
```

```
def funcion(a, b, c):  
    print(a, b, c)
```

```
funcion(1, a=2, b=3)    # error por rep. a
```

```
funcion(1, c=2, 3)      # error 3er arg
```

```
def funcion2(a, b=2, c, d): # error  
    print(a, b, c, d)
```



# Valor por defecto

Los valores por defecto se evalúan UNA ÚNICA VEZ en la definición de la función.

```
def agrego(a, lista=[]):
```

```
    lista.append(a)
```

```
    return lista
```

```
print(agrego(1)) # [1]
```

```
print(agrego(2)) # [1, 2]
```

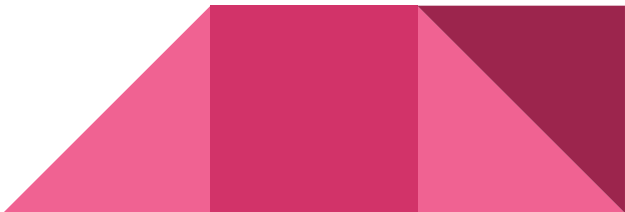
```
print(agrego(3)) # [1, 2, 3]
```



# Cantidad de parámetros indeterminado (\*args, \*\*kwargs)

```
def funcion(a, b, *args, **kwargs):  
    print(a, b, end=' ')  
    for elem in args:  
        print(elem, end=' ')  
    for key in kwargs:  
        print(key, kwargs[key], end=' ')  
    print('')
```

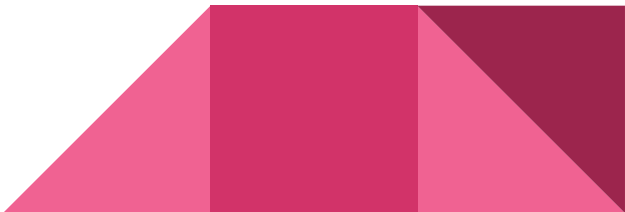
```
funcion(1, 2, 3, 4, 5, seis=6, siete=7) # 1 2 3 4 5 seis 6 siete 7  
funcion(1, 2)                          # 1 2  
funcion(1, 2, 3, 4, 5)                  # 1 2 3 4 5  
funcion(1, 2, seis=6, siete=7)          # 1 2 seis 6 siete 7
```



# Desempaquetar argumentos

```
def funcion(*args, **kwargs):  
    for elem in args:  
        print(elem, end=' '  
    for key in kwargs:  
        print(key, kwargs[key], end=' '  
    print(' '  
  
lista = [1, 2, 3]  
dic = {'cuatro': 4, 'cinco': 5}  
  
funcion(*lista, **dic) # 1 2 3 cuatro 4 cinco 5
```

```
def funcion2(a, b, c):  
    print(a, b, c)  
  
lista2 = [1, 2, 3]  
# len(lista2) == cant. param.  
  
dic2 = {'a': 1, 'b': 2, 'c': 3}  
# keynames == param.  
  
funcion2(*lista2) # 1 2 3  
funcion2(*dic2)  # a b c  
funcion2(**dic2) # 1 2 3
```





# Ejercicio ejemplo

Queremos definir una función que, dada una cadena de caracteres, retorne la cantidad de vocales abiertas (a, e, o), vocales cerradas (i, u) y la cantidad total de caracteres de la misma.



# Resolución

```
def retorno_varios(cadena):  
    """Esta función retorna la cantidad de vocales abiertas, cerradas y el tamaño de una cadena recibida  
    como parámetro."""  
    cadena = cadena.lower()  
    cant_aeo = cadena.count("a") + cadena.count("e") + cadena.count("o")  
    cant_iu = cadena.count("i") + cadena.count("u")  
  
    return cant_aeo, cant_iu, len(cadena)  
  
resultado = retorno_varios("La argentinidad al palo.")  
print(type(resultado)) # tuple  
print(resultado[2]) # 24  
  
abiertas, cerradas, longitud = retorno_varios("La argentinidad al palo.")
```



# Atributos de funciones

- `funcion.__doc__` : es el Docstring
- `funcion.__name__` : es una cadena con el nombre de la función.
- `funcion.__defaults__` : es una tupla con los valores por defecto de los parámetros opcionales.



# Ejercicio 1

Escribir una función que reciba un número entero positivo y devuelva su factorial.

$$6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720$$



## Ejercicio 2

Escribir una función que calcule el total de una factura tras aplicarle el IVA. La función debe recibir la cantidad sin IVA y el porcentaje de IVA a aplicar, y devolver el total de la factura. Si se invoca la función sin pasarle el porcentaje de IVA, deberá aplicar un 21%.



## Ejercicio 3

Escribir una función que calcule el área de un círculo y otra que calcule el volumen de un cilindro usando la primera función (`from math import pi`).

$\text{Area} = \pi * \text{radio}^2$

$\text{Volumen} = \text{Area} * \text{altura}$



## Ejercicio 4

Escribir una función que reciba una muestra de números en una lista y devuelva el promedio.



## Ejercicio 5

Escribir una función que reciba una muestra de números en una lista y devuelva otra lista con sus cuadrados.





## Ejercicio 6

Escribir un programa que reciba una cadena de caracteres y devuelva un diccionario con cada palabra que contiene y su frecuencia. Escribir otra función que reciba el diccionario generado con la función anterior y devuelva una tupla con la palabra más repetida y su frecuencia.

Ej:

```
text = 'Como quieres que te quiera si el que quiero que me quiera no me quiere como  
quiero que me quiera'
```

```
{'Como': 1, 'quieres': 1, 'que': 4, 'te': 1, 'quiera': 3, 'si': 1, 'el': 1, 'quiero': 2, 'me': 3, 'no': 1,  
'quiere': 1, 'como': 1}
```

```
('que', 4)
```

